

Transform and Conquer

Dönüştür ve Yönet Algoritma Tasarım Tekniği

Transform and Conquer ? (Brute Force, Divide&Conquer, Dynamic Programming, Greedy, Transform and Conquer)

- Beşinci algoritma tasarım tekniği
- Üç ana varyasyon:
 - **Basitleştirme:** Aynı problemin daha basit yada daha uygun bir örneğine dönüştürme
 - **Gösterim değişikliği:** Aynı örneğin farklı bir **gösterimine** dönüştürme
 - **Problem dönüştürme:** Etkili bir algoritma ile çözebildiğiniz farklı bir problem örneğine dönüştürün
- İki adımlı süreç:
 - **Adım 1:** Problemi daha kolay çözülebilecek bir şekilde uyarlayın
 - **Adım 2:** Yönetin!

Transform and Conquer ?

1. ÖnceSırala

Problem
Örneği

Daha basit problem
yada

Diğer bir gösterim
yada

Diğer bir problem örneğine
indirgeme

Çözüm

1. Heapsort
2. Horner Yöntemi
3. Binary Exponentiation

1. En Küçük Ortak Çoklu Hesaplama
2. Graftaki yolları hesaplama
3. Optimizasyon Problemlerine dönüşüm
4. Graf problemlerine dönüşüm

Trns. & Conq.: Presorting

- Liste sıralanırsa bir liste hakkında birçok sorunun yanıtlanması daha kolaydır.
- Örnek 1:Dizideki öge benzersizliğini kontrol etme

4	1	8	9	3	7	10	2	3	1
---	---	---	---	---	---	----	---	---	---

$$T(n) = (n-1) + (n-2) + \dots + 1 \\ = \frac{(n-1)n}{2} \in \Theta(n^2)$$

Brute-force?

İlk elemanı seç ve dizinin geri kalanında olup olmadığını kontrol et.
İkinci elemanı seç ve dizinin geri kalanında olup olmadığını kontrol et

Trns. & Conq.: Presorting

- Önce-sırala'yı uygularsak

ALGORITHM PresortElementUniqueness(A[0..n-1])

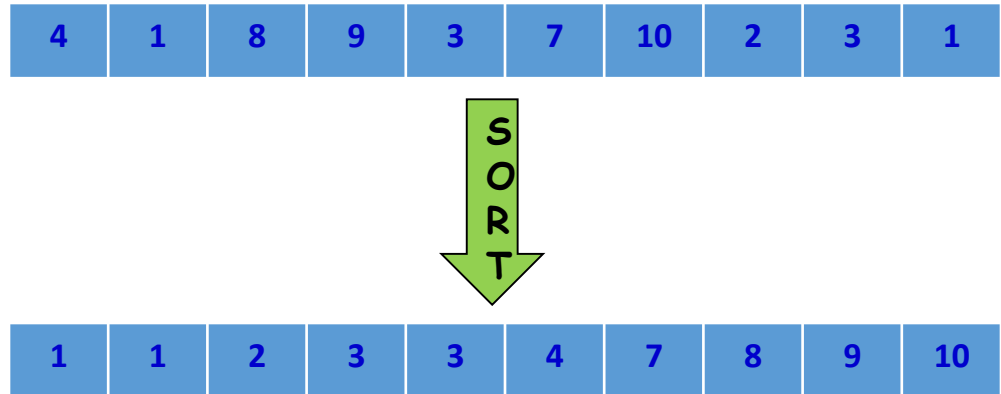
sort the array A

for i <- 0 **to** n-2 **do**

if A[i] = A[i+1]

return false

return true



$$T(n) = T_{\text{sort}}(n) + T_{\text{scan}}(n) \in \Theta(n \lg n) + \Theta(n) = \Theta(n \lg n)$$

Trns. & Conq.: Önce Sırala

- Örnek 2: Mod Hesaplama
- Mod, listede en sık tekrarlanan değerdir?

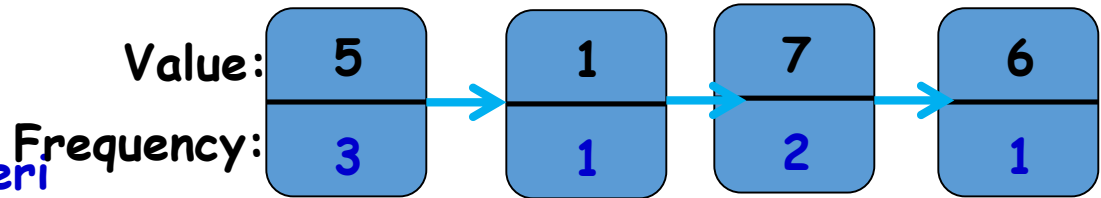
5	1	5	7	6	5	7
---	---	---	---	---	---	---

5

Brute-force: listeyi tara ve tüm farklı değerlerin frekanslarını hesapla, ardından en yüksek frekansa sahip değeri bulun

Brute-force
nasıl işletilecek?

Daha önce karşılaşılan değerleri frekanslarıyla birlikte ayrı bir listede saklayın.



Enkötü durum girişi nedir?

$$C(n) = 0 + 1 + \dots + (n-1) \\ = \frac{(n-1)n}{2} \in \Theta(n^2)$$

Hiç eşit değeri olmayan bir dizi

Trns. & Conq.: Önce sırala

- Örnek 2-devam: İlk önce diziyi sıralarsak, tüm eşit değerler birbirine bitişik olur. Modu hesaplamak için bilmemiz gereken tek şey, sıralanmış dizideki bitişik eşit değerlerin en uzun çalışmasını bulmaktır.

Trns. & Conq.: Presorting

Örnek 2: Computing a mode (contd.)

ALGORITHM PresortMode(A[0..n-1])

sort the array A

i <- 0

modelfrequency <- 0

while i ≤ n-1 **do**

 runlength <- 1; runvalue <- A[i]

while i+runlength ≤ n-1 **and** A[i+runlength] = runvalue

 runlength <- runlength+1

if runlength > modelfrequency

 modelfrequency <- runlength; modevalue <- runvalue

 i <- i+runlength

return modevalue

1	1	2	3	3	3	7	8	9	10
---	---	---	---	---	---	---	---	---	----

while loop doğrusal zaman maliyetindedir,
genel çalışma süresini, sıralama zamanı maliyeti belirler,
 $\Theta(n \lg n)$

Trns. & Conq.: Heapsort

- Bir diğer $O(n \lg n)$ sıralama algoritması
- “(Heap)” adı verilen akıllı bir veri yapısı kullanır
- Bir diziyi bir Heap’e dönüştürür ve sonra sıralama çok kolay olur.
- Heap, “priority queue” gibi önemli bir kullanıma sahiptir.
- Öncelikli kuyruk, aşağıdaki işlemlerle “öncelik” adı verilen düzenlenebilir bir işletim oluşturur.
 - **Find** :Enyüksek öncelikli elemanı bulmak
 - **Deleting** Enyüksek öncelikli elemanı silmek
 - **Adding**: Çoklu kümeye yeni bir eleman eklemek

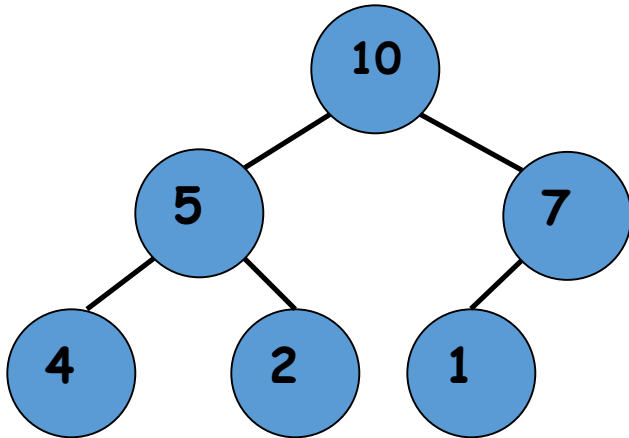
Trns. & Conq.: Heap

- Bir “yığın”, aşağıdaki iki koşulun sağlanması şartıyla düğüm başına bir anahtarla ikili ağaç olarak tanımlanabilir :
 - **Şekil Özelliği**: İkili ağaç tam olarak doludur, son seviye hariç bütün seviyeler doludur, sadece sağdan bazı yapraklar boş olabilir.
 - **Ebeveyn kontrolü** (heap özelliği): Herbir düğümdeki anahtar çocuklarının anahtarlarından büyük yada eşittir.

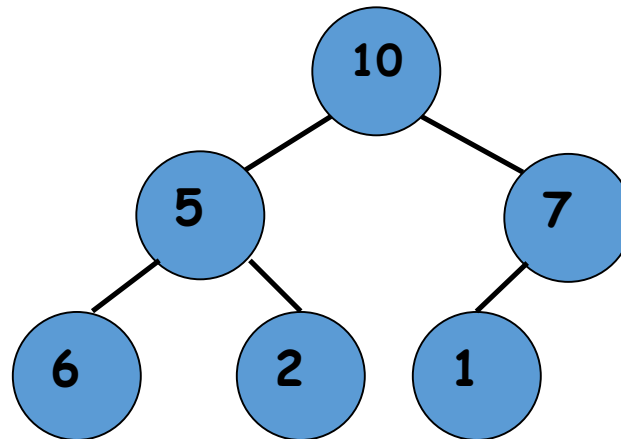
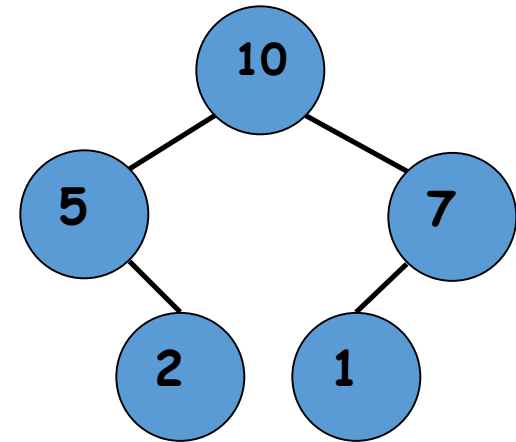
Bu “max heap” tanımıdır. Uygun düzenleme ile “min heap” tanımı da yapılabilir.

Trns. & Conq.: Heap

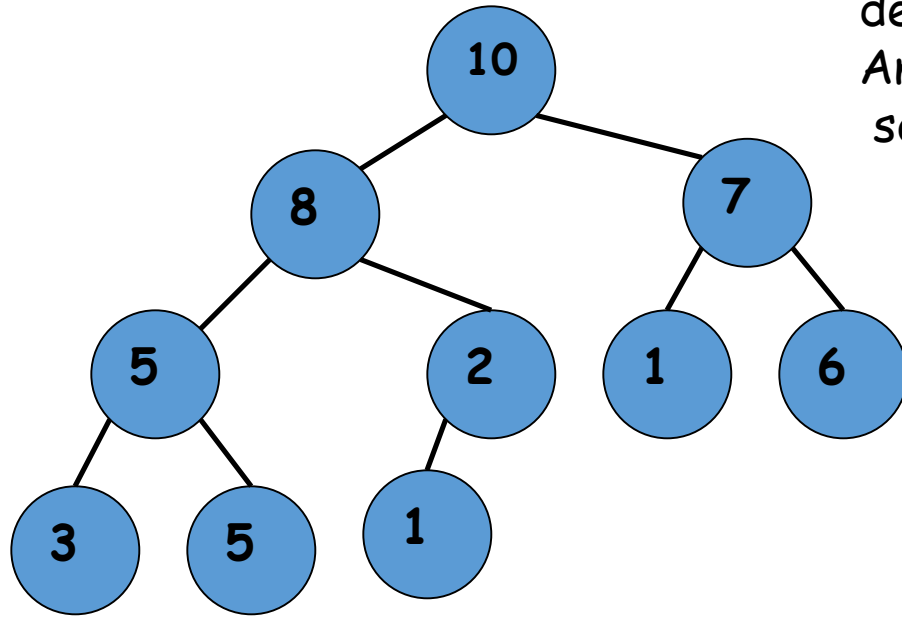
Şekil ve Heap Özellikleri



Heap ?



Trns. & Conq.: Heap

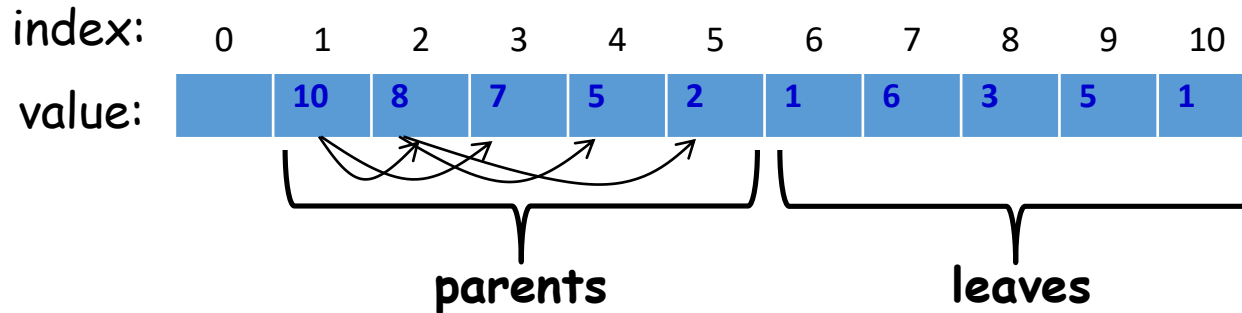


Dikkat:
Kökten bir yaprağa giden yoldaki
değerler dizisi artansırada değildir.
Anahtar değerleri, yukarıdan aşağıya ve
soldan sağa yerleştirilir.

ALGORITHM Parent(i)
return $\lfloor i/2 \rfloor$

ALGORITHM Left(i)
return $2i$

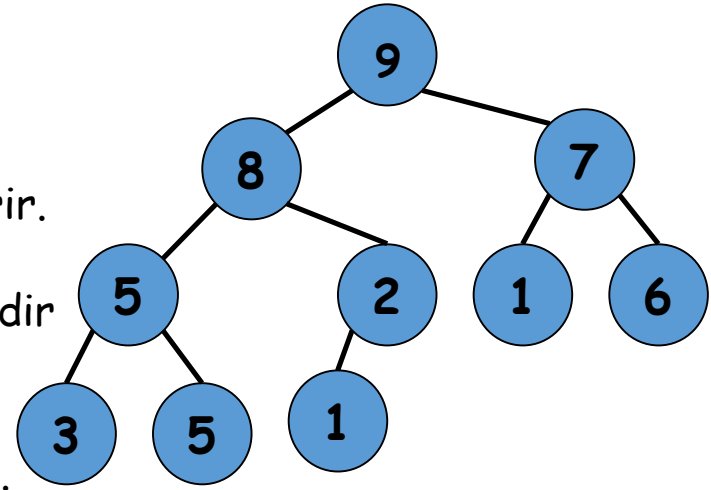
ALGORITHM Right(i)
return $2i + 1$



Trns. & Conq.: Heap

Important properties of heaps:

1. n düğümlü tam bir ikili ağaçtır, yüksekliği: $\lceil \lg n \rceil$
2. Heap ağacının kök düğümü en büyük elemanı içerir.
3. Bir düğümün bütün torunları da heap özelliğindedir
4. Heap bir dizi olarak işletilebilir.



Elemanları topdown ve left-right tarzda kaydedilir.

$H[0]$ kullanılmaz yada sağdakilerden daha büyük sayıyı içerir.

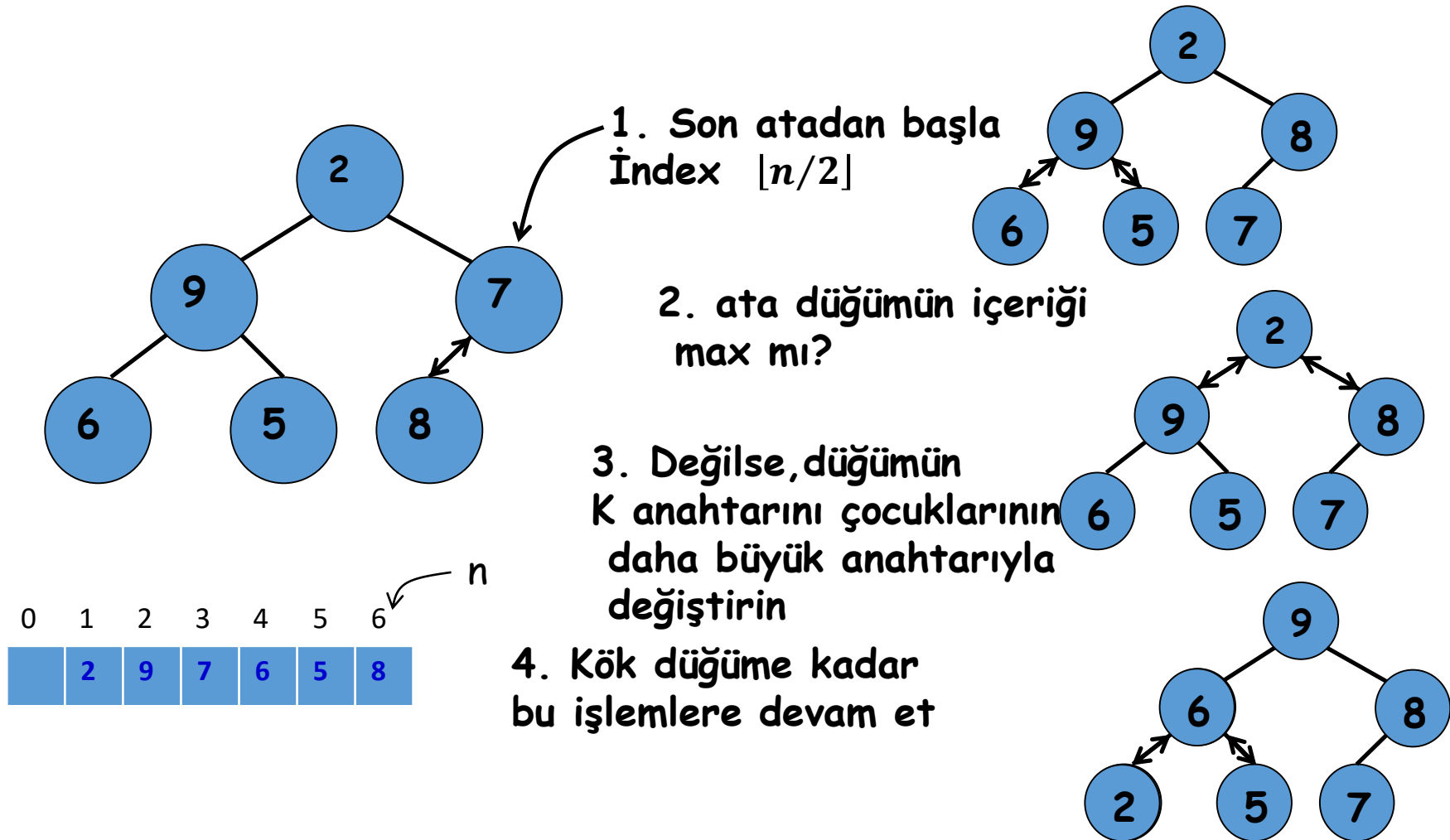


- a. Ebeveyn düğüm ilk $\lfloor n/2 \rfloor$ pozisyonunda ve yapraklar son $\lfloor n/2 \rfloor$ pozisyonunda olacaktır.
- b. i ($1 \leq i \leq \lfloor n/2 \rfloor$) indeksindeki elemanın çocukları $2i$ ve $2i+1$ pozisyonlarda olacaktır. i ($2 \leq i \leq n$) indeksinin atası ise $\lfloor i/2 \rfloor$ pozisyonunda olacaktır.

**Bu durumda : $H[i] \geq \max\{ H[2i], H[2i+1] \}$
for $i = 1, \dots, \lfloor n/2 \rfloor$**

Trns. & Conq.: Heap

- Bir diziyi heap ağacına nasıl dönüştürebiliriz. ? İki yoldan:



Bu sürece "Heapifying" adı verilir.

Bu metot "bottom-up heap construction" olarak anılır.

Trns. & Conq.: Heap

ALGORITHM HeapBottomUp($H[1..n]$)

//Input: $H[1..n]$ dizisi.

//Output: Heap $H[1..n]$

for $i \leftarrow \lfloor n/2 \rfloor$ **downto** 1 **do**

$k \leftarrow i$; $v \leftarrow H[k]$

 heap $\leftarrow$ **false**

while not heap and $2*k \leq n$ **do**

$j \leftarrow 2*k$

if $j < n$ //2 çocuk var

if $H[j] < H[j+1]$

$j \leftarrow j+1$

if $v \geq H[j]$ // sadece sol çocuk

 heap $\leftarrow$ **true**

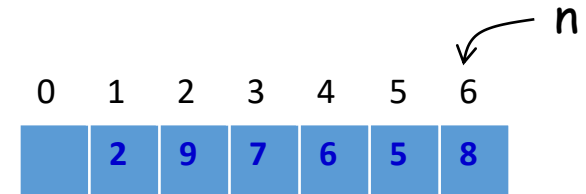
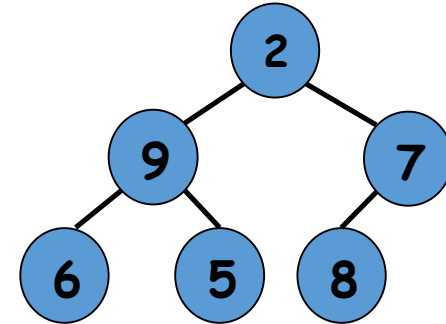
else

$H[k] \leftarrow H[j]$

$k \leftarrow j$

$H[k] \leftarrow v$

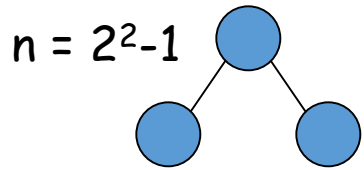
Heapify
Bir ebeveyn için



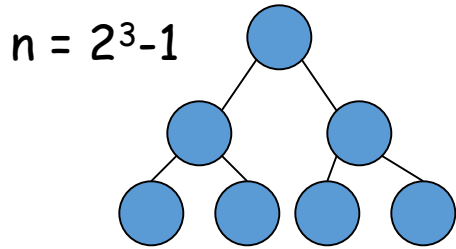
Trns. & Conq.: Heap

- HeapBottomUp(H[1..n]) için en kötü durum analizi:

Tam dolu bir ikili heap ağacı alalım.



Düğüm sayısı, $n = 2^m - 1$



Ağacın yüksekliği ?

$$h = \lfloor \lg n \rfloor = \lfloor \lg(n + 1) \rfloor - 1 = m - 1$$

İ seviyesindeki bir key yaprak seviyesi h kadar gezmeli; i seviyesi (h-i) alt seviyeye sahiptir. 1 seviyeye inmek 2 karşılaştırma gerektirir: Biri daha büyük çocuğu belirlemek için diğeri ise değişim gerekip gerekmediğini belirlemek için.

Bu nedenle, 2n'den az

karşılaştırma

yapılması gerekir

$$C_{\text{worst}}(n) = \sum_{i=0}^{h-1} \sum_{j=1}^{2^i} 2(h-i)$$

$$= \sum_{i=0}^{h-1} 2(h-i)2^i$$

$$= 2h \sum_{i=0}^{h-1} 2^i - 2 \sum_{i=0}^{h-1} i2^i$$

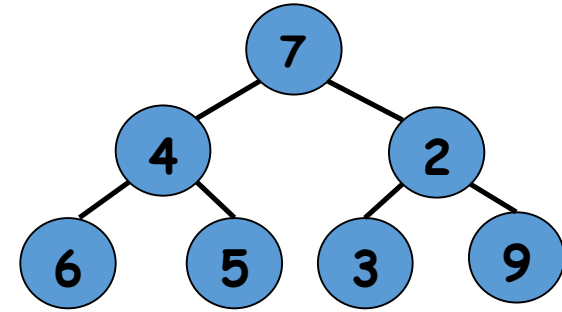
$$= 2h(2^h - 1) - 2(h-2)2^h + 4$$

$$= h2^{h+1} - 2h - h2^{h+1} + 2 \cdot 2^{h+1} + 4$$

$$= 2(n - \lg(n+1) + 4)$$

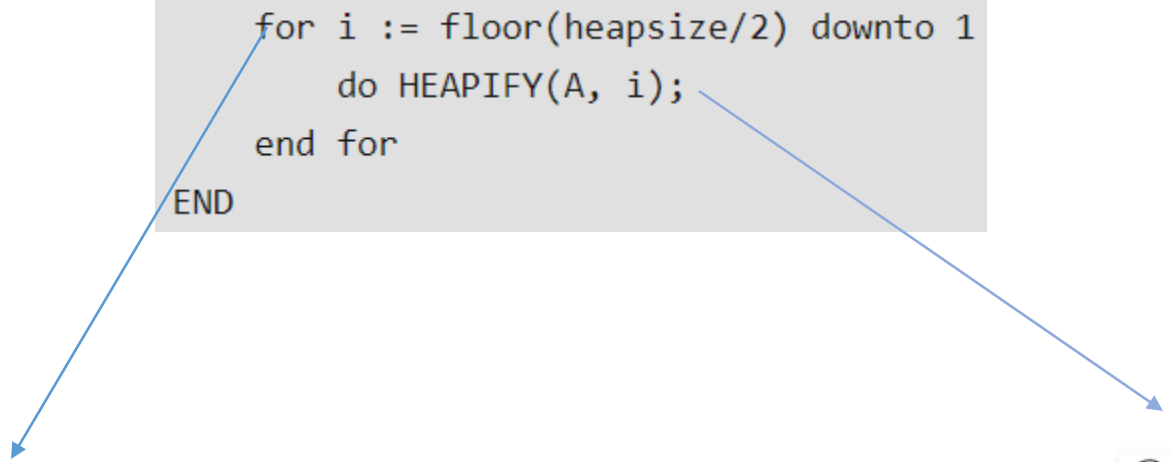
$$\sum_{i=1}^n i2^i = (n-1)2^{n+1} + 2$$

Neden?



worst-case...


```
BUILD-HEAP(A)
  heapsize := size(A);
  for i := floor(heapsize/2) downto 1
    do HEAPIFY(A, i);
  end for
END
```

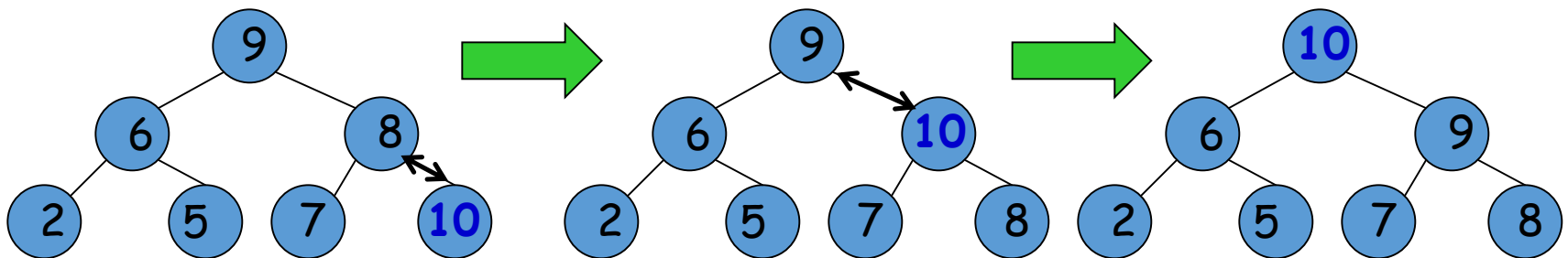


$O(n)$

$O(\lg(n))$

Trns. & Conq.: Heap

- Heap yapılandırması, ikinci yol:
- “top-down heap construction”
- Idea: Önceden oluşturulmuş bir öbeğe başarıyla yeni bir anahtar ekleyin
- Mevcut yığının son yaprağından sonra K anahtarıyla yeni bir düğüm ekleyin
- Heap özelliği sağlanana kadar K'yi yukarı kaldırın

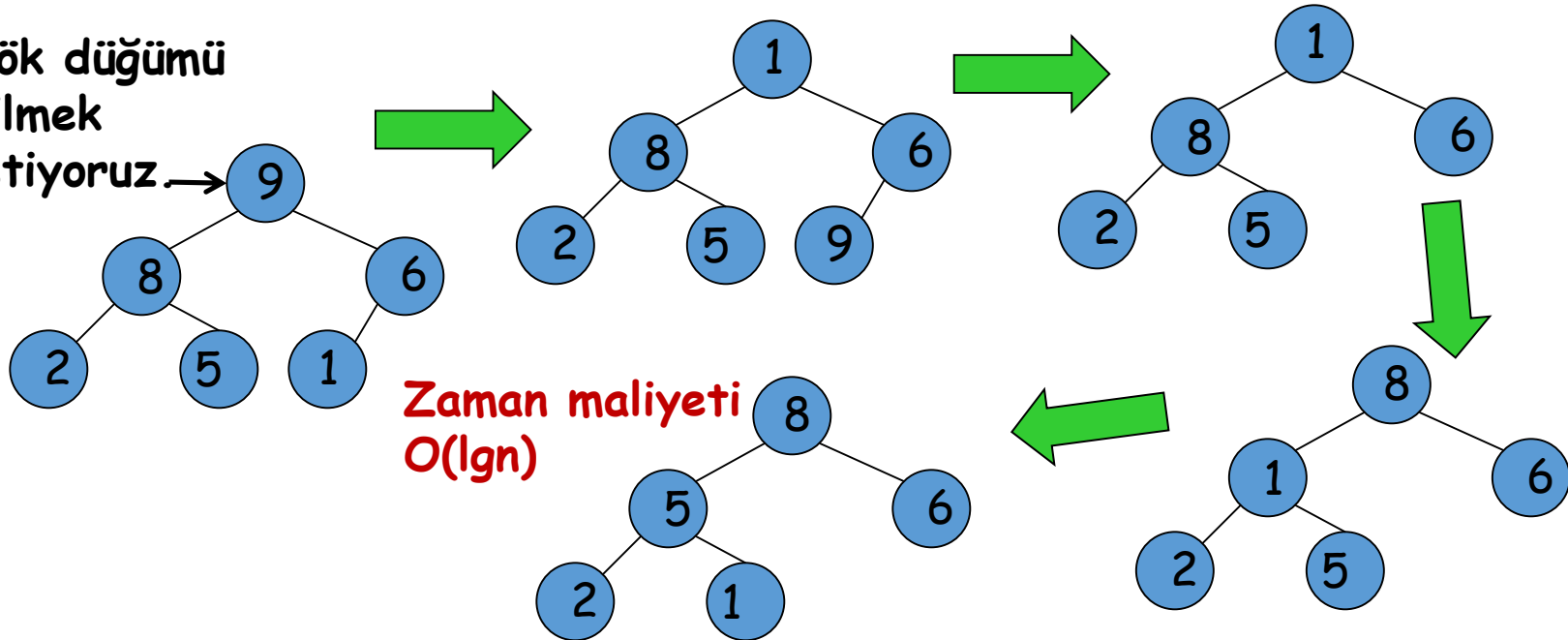


Ekleme işlemi, öbek ağacının yüksekliğinin iki katından fazlasını alamaz, bu nedenle çalışma zamanı, $O(\lg n)$

Trns. & Conq.: Heap

- Bir heap ağacından max key'i silmek
 - Kökün anahtarını heap'deki son K anahtarıyla değiştirin
 - Heap eleman sayısını 1 azaltın
 - Gerekli olduğu sürece K'yi kaydırarak "heap özelliği" oluşturarak "heapleştir"

Kök düğümü
silme
istiyoruz →



Trns. & Conq.: Heapsort

- İki aşamalıdır
 - Adım 1: Verilen diziden bir heap ağacı oluştur (Heapify)
 - Adım 2: Kök silme işlemini $n-1$ defa uygula

Trns. & Conq.: Heapsort

Aşama 1: Heap yapılandırması

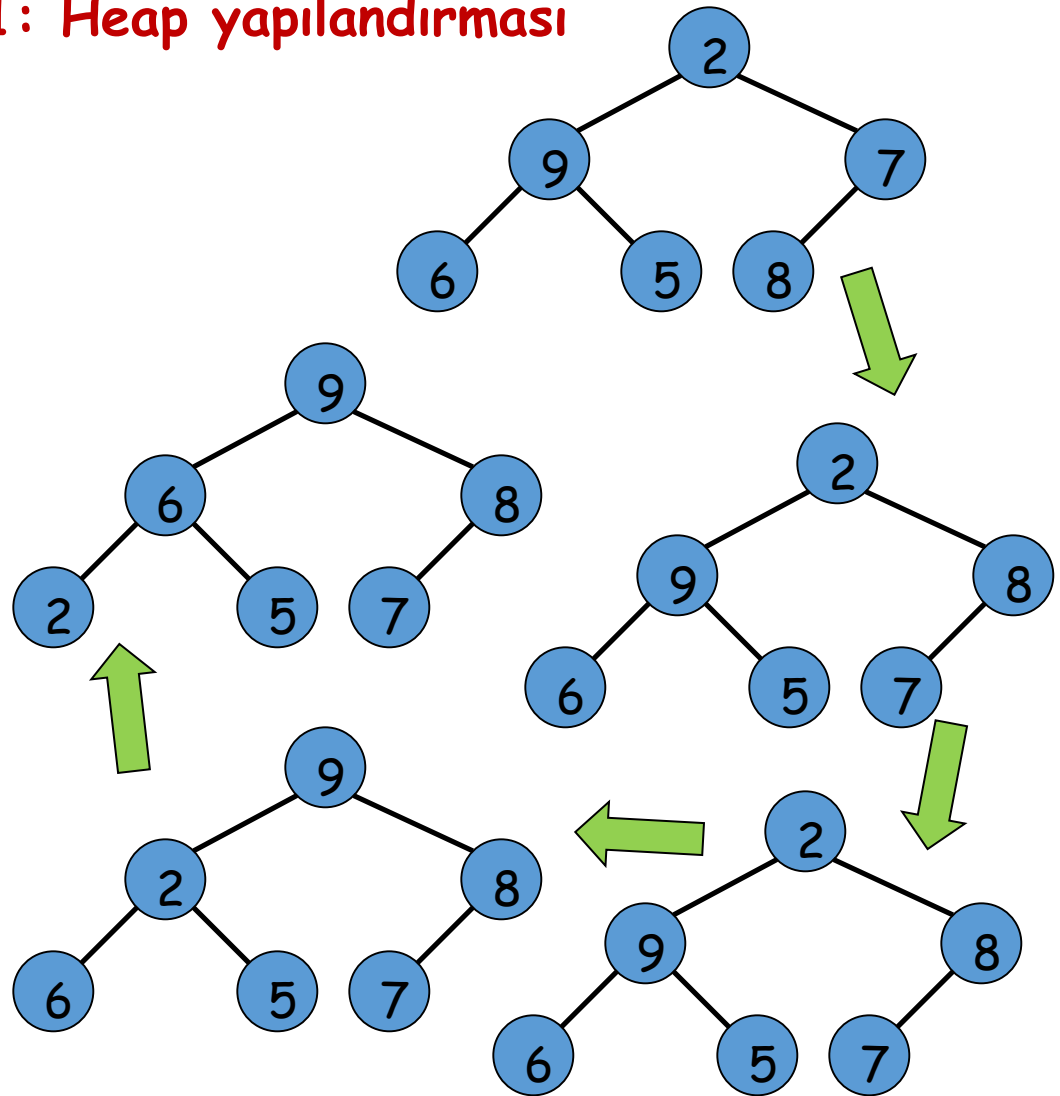
0	1	2	3	4	5	6
	2	9	<u>7</u>	6	5	<u>8</u>

0	1	2	3	4	5	6
	2	<u>9</u>	8	<u>6</u>	<u>5</u>	7

0	1	2	3	4	5	6
	<u>2</u>	<u>9</u>	<u>8</u>	6	5	7

0	1	2	3	4	5	6
	9	2	8	6	5	7

0	1	2	3	4	5	6
	9	6	8	2	5	7



Trns. & Conq.: Heapsort

Aşama 2: Maximumu sil

0	1	2	3	4	5	6
	9	6	8	2	5	7

0	1	2	3	4	5	6
	7	6	8	2	5	9

0	1	2	3	4	5	6
	8	6	7	2	5	9

0	1	2	3	4	5	6
	5	6	7	2	8	9

0	1	2	3	4	5	6
	7	6	5	2	8	9

0	1	2	3	4	5	6
	2	6	5	7	8	9

0	1	2	3	4	5	6
	6	2	5	7	8	9

0	1	2	3	4	5	6
	5	2	6	7	8	9

0	1	2	3	4	5	6
	2	5	6	7	8	9

Trns. & Conq.: Heapsort

- Heapsort'un en kötü durum karmaşıklığı nedir?

Aşama 1: Heap yapılandırması? $O(n)$

Aşama 2: Max içerikli düğümü sil? $?...$

$C(n)$: Aşama 2'deki karşılaştırma sayısı olsun

Heap ağacının yüksekliği: $\lfloor \lg n \rfloor$

$$C(n) \leq 2\lfloor \lg(n-1) \rfloor + 2\lfloor \lg(n-2) \rfloor + \dots + 2\lfloor \lg(1) \rfloor \leq 2\sum_{i=1}^{n-1} \lg(i)$$

$$C(n) \leq 2 \sum_{i=1}^{n-1} \lg(n-i) = 2(n-1)\lg(n-1) \leq 2n \lg n \quad \text{So, } C(n) \in O(n \lg n)$$

Bu nedenle, aşama 2 için, $O(n) + O(n \lg n) = O(n \lg n)$

Trns. & Conq.: Heapsort

- Heapsort'un en iyi ve en kötü durumu Mergesort'da olduğu gibi $\Theta(n \lg n)$ olmaktadır.
- Heapsort ekstra bellek kullanmaz.
- Random dosyalar üzerindeki zamanlama deneyleri, Heapsort'un Quicksort'tan daha yavaş olduğunu, ancak Mergesort ile rekabet edebileceğini göstermektedir.

Polinom Hesabı

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

Brute Force ?

Adım1 (KOŞULLAMA)

P=X ve k=1

Adım 2 (SONRAKİ KUVVET)

$$X^2 = X * X,$$

While k < n

$$X^3 = (X^2) * X$$

(a) $P \leftarrow P.X$

...,

(b) $k \leftarrow k+1$

$X^n = X^{n-1} * X$ olduğuna göre toplam

EndWhile

n-1 çarpma gerekecektir.

Adım 3: ($P = X^n$ olur)

Print P.

Analiz: Dikkat edilirse 2.adımda $n-1$ çarpma $n-1$ toplama ve adet karşılaştırma ($k=n$ olduğunda 2.adımdan çıkılıyor) yapılıyor.

Buna göre X^n in hesaplanması için toplam $(n-1)+(n-1)+n=3n-2$ adet elemanter işlem yapılmaktadır.

Genellikle , bu işlem sayısında tam bir değer değil de bir merteye söylememiz bizim için yeterlidir. Bu örnek için "işlem sayısı **$3n$** civarındadır".

Hatta "işlem sayısı n in bir sabitle çarpımı kadardır" bile diyebiliriz.

Çoğunlukla n in büyümesi durumunda işlem sayısının hızla büyüüp büyümediği bizi daha çok ilgilendirir.

Adım1 (KOŞULLAMA)

$P=X$ ve $k=1$

Adım 2 (SONRAKİ KUVVET)

While $k < n$

(a) $P \leftarrow P.X$

(b) $k \leftarrow k+1$

EndWhile

Adım 3: ($P=X^n$ olur)

Print P .

$P(X)=a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$; burada $a_n \neq 0$ ve a_i
($i=0,1,\dots,n$) sabit

$P(X)=a_n x^n + \dots + a_0$, n pozitif tam sayı, $x, a_0, \dots, a_n$ reel sayılar

Adım 1: (Koşullama)

$S = a_0, k=1$

Adım 2 : (Bir sonraki terimi ekle)

While $k \leq n$

(a) $S \leftarrow S + a_k x^k$

(b) $k \leftarrow k+1$

EndWhile

Adım 3: Print s

Analiz: 2.adımda $k \leq n$ kontrolünü $k=1,2,\dots,n+1$ için $n+1$ kere yapıyoruz .

Herhangi bir $k \leq n$ için 1 karşılaştırma, 2 toplama, 1 çarpma ve $(3k-2)$ işlem x^k hesabı için yapıyoruz.

O halde $3k-2+4=3k+2$ işlem yapılmaktadır.

$k=1,2,\dots,n$ için $5+8+11+\dots+3n+2=(3n^2 + 7n)/2$ işlem yapılıyor.

$k=n+1$ için yapılan karşılaştırmayı da eklersek $(3n^2 + 7n)+1) / 2$ işlem yapılmaktadır.

Görüldüğü gibi algoritmanın karmaşıklılığı bir polinomdur , yani $1.5n^2+3.5n+0.5$ dir. İşlem yükünün hesabında n^2 terimi daha etkin olacağından Bu algoritmanın karmaşıklığı $O(n^2)$ olarak ifade edilir.

Transform & Conquer (Horner Yöntemi)

$$a_3x^3 + a_2x^2 + a_1x + a_0 = x(a_3x^2 + a_2x + a_1) + a_0 = x(x(a_3x + a_2) + a_1) + a_0 = x(x(a_3(x) + a_2) + a_1) + a_0$$

$$a_3(x)$$

Adım 1 $S = a_n$ ve $k=1$

$$a_3(x) + a_2$$

Adım 2 While $k \leq n$

(a) $S \leftarrow XS + a_{n-k}$

(b) $k \leftarrow k+1$

EndWhile

$$x(a_3(x) + a_2) + a_1$$

$$x(x(a_3(x) + a_2) + a_1) + a_0$$

Adım 3 Print s .

Analiz: Adım 2 de her $k \leq n$ için 1 karşılaştırma, 1 çarpma, 2 toplama, 1 çıkarma yapılmaktadır. n için $5n+1$ işlem yapılmaktadır.

Önceki algoritma ile karşılaştırsak, örneğin $n=10$ için 51 işleme karşı 186 işlem olacaktır. n 'in büyük değeri için fark çok daha artacaktır.

Horner yönteminin karmaşıklılığı $O(n)$ olduğu için Brute Force tasarım tekniğine göre daha performans üstünlüğü içeren bir algoritmadır.

“Highest to Lowest Term” Brute-Force Algorithm

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

$$p \leftarrow 0$$

for $i \leftarrow n$ **downto** 0 **do**

$$\text{power} \leftarrow 1$$

for $j \leftarrow 1$ **to** i **do**

$$\text{power} \leftarrow \text{power} * x \text{ // compute } x^i \text{ (} 1 \leq i \leq n \text{)}$$

$$p \leftarrow p + a[i] * \text{power} \text{ // compute } a_i x^i$$

return p

$$\begin{aligned} M(n) &= \sum_{i=0}^n \left(\sum_{j=1}^i 1 + 1 \right) = \sum_{i=0}^n (i+1) \\ &= \sum_{i=0}^n i + \sum_{i=0}^n 1 = \frac{n(n+1)}{2} + (n+1) \\ &= \frac{(n+1)(n+2)}{2} \in \Theta(n^2). \end{aligned}$$

“Lowest to Highest Term” Brute-Force Algorithm

$p \leftarrow a[0]; power \leftarrow 1$

for $i \leftarrow 1$ **to** n **do**

$power \leftarrow power * x$

$p \leftarrow p + a[i] * power$

return p

$$M(n) = \sum_{i=1}^n 2 = 2n \in \Theta(n)$$

Horner's Rule ile Polynomial değeri Hesaplama

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

$$p(x) = (a_n x^{n-1} + a_{n-1} x^{n-2} + \dots + a_1) x + a_0$$

...

$$p(x) = (\dots (a_n x + a_{n-1}) x + \dots) x + a_0.$$

$$\begin{aligned} p(x) &= 2x^4 - x^3 + 3x^2 + x - 5 \\ &= (2x^3 - x^2 + 3x + 1)x - 5 \\ &= ((2x^2 - x + 3)x + 1)x - 5 \\ &= (((2x - 1)x + 3)x + 1)x - 5. \end{aligned}$$

$$M(n) = n = 4$$

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

$$p(x) = (\dots (a_n x + a_{n-1})x + \dots)x + a_0.$$

$$p(x) = 2x^4 - x^3 + 3x^2 + x - 5$$

$$p(x) = 2x^4 - x^3 + 3x^2 + x - 5 = x(2x^3 - x^2 + 3x + 1) - 5 = x(x(2x^2 - x + 3) + 1) - 5 = x(x(x(2x - 1) + 3) + 1) - 5.$$

$$p(x) = 2x^4 - x^3 + 3x^2 + x - 5 \quad x = 3.$$

Katsayılar	2	-1	3	1	-5
$x = 3$	2	$3 \cdot 2 + (-1) = 5$	$3 \cdot 5 + 3 = 18$	$3 \cdot 18 + 1 = 55$	$3 \cdot 55 + (-5) = 160$

ALGORITHM *Horner*($P[0..n]$, x)

//Input: $P[0..n]$ n dereceli polinomun katsayılarını tutan dizi

//Output: x noktasında polinomun değeri

$p \leftarrow P[n]$

for $i \leftarrow n - 1$ **downto** 0 **do**

$p \leftarrow x * p + P[i]$

return p

Toplama ve
çarpımların toplamı:

$$M(n) = A(n) = \sum_{i=0}^{n-1} 1 = n.$$

Horner's Rule to Compute a^n

- Brute-force algorithm

$$a^n = a \times a \times \dots \times a. \quad M(n) = n - 1 \in \Theta(n) \approx \Theta(2^b)$$

b:n'in ikili
gösterimindeki bit
sayısı

Horner's Rule to Compute a^n

Divide-and-conquer algorithm

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ a & \text{if } n = 1 \\ a^{\lfloor n/2 \rfloor} \cdot a^{\lceil n/2 \rceil} & \text{if } n > 1 \end{cases} \quad a^5 = a^2 \cdot a^3$$

Algorithm *PowerDC*(a, n)

if $n = 1$ **return** a

else return *PowerDC*($a, \lfloor n / 2 \rfloor$) * *PowerDC*($a, \lceil n / 2 \rceil$)

$$M(n) = M(\lfloor n / 2 \rfloor) + M(\lceil n / 2 \rceil) + 1 \text{ for } n > 1$$

$$M(1) = 0.$$

$$M(n) = n - 1 \in \Theta(n) \approx \Theta(2^b). \quad n = 2^k$$

$$a^n = \begin{cases} (a^{n/2})^2 & \text{if } n \text{ is even and positive} \\ (a^{(n-1)/2})^2 \cdot a & \text{if } n \text{ is odd and greater than 1} \\ a & \text{if } n = 1. \end{cases}$$

- $a^4 = (a^2)^2$
 - $a^5 = (a^2)^2 \cdot a$
- $M(n) = M(n / 2) + c$ for $n > 1$, $c = 1$ or 2 , $M(1) = 0$.

Master Theorem,

$$a = 1, b = 2, d = 0. \quad a = b^d.$$

$$M(n) \in \Theta(\log n) \approx \Theta(b).$$

representation change

Binary Exponentiation

$$n = b_I \dots b_i \dots b_0$$

$$p(x) = b_I x^I + \dots + b_i x^i + \dots + b_0$$

$$13 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0.$$

$$a^n = a^{p(2)} = a^{b_I 2^I + \dots + b_i 2^i + \dots + b_0}.$$

Horner's rule $p(2)$	$a^n = a^{p(2)}$
$p \leftarrow 1$ //the leading digit is always 1 for $n \geq 1$ for $i \leftarrow I - 1$ downto 0 do $p \leftarrow 2p + b_i$	$a^p \leftarrow a^1$ for $i \leftarrow I - 1$ downto 0 do $a^p \leftarrow a^{2p+b_i}$

$$a^{2p+b_i} = a^{2p} \cdot a^{b_i} = (a^p)^2 \cdot a^{b_i} = \begin{cases} (a^p)^2 & \text{if } b_i = 0, \\ (a^p)^2 \cdot a & \text{if } b_i = 1. \end{cases}$$

ALGORITHM *LeftRightBinaryExponentiation*($a, b(n)$)

// left-to-right binary exponentiation algorithm ile a^n hesaplanır

//Input: a ve n pozitif sayısının ikili açılımındaki

// ikili dijitalerin $b_I, \dots, b_0$ listesi

//Output: a^n değeri

$product \leftarrow a$

for $i \leftarrow I - 1$ **downto** 0 **do**

$product \leftarrow product * product$

if $b_i = 1$ $product \leftarrow product * a$

return $product$

$$a^{13} \quad n = 13 = 1101_2.$$

$$13 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0.$$

n 'in ikili digitleri

çarpımlar

1

1

0

1

a

$a^2 \cdot a = a^3$

$(a^3)^2 = a^6$

$(a^6)^2 \cdot a = a^{13}$

Algoritma tek döngüsünün her tekrarında bir veya iki çarpma yaptığından

$$(b - 1) \leq M(n) \leq 2(b - 1)$$

$M(n)a^n$ hesabındaki çarpma sayısı, b ise bit dizisinin uzunluğudur.

Bu nedenle, bu algoritma her zaman $n - 1$ çarpımları gerektiren kaba kuvvet üssünden daha iyi bir verimlilik sınıfındadır.

$b - 1 = \lfloor \log_2 n \rfloor$. Verimlilik logaritmiktir.

b : n 'in ikili gösteriminin bir uzunluğudur.

Right-to-left binary exponentiation

$$a^n = a^{b_I 2^I + \dots + b_i 2^i + \dots + b_0} = a^{b_I 2^I} \cdot \dots \cdot a^{b_i 2^i} \cdot \dots \cdot a^{b_0}$$

$$a^{2^i} = (a^{2^{i-1}})^2$$

$$a^{b_i 2^i} = \begin{cases} a^{2^i} & \text{if } b_i = 1, \\ 1 & \text{if } b_i = 0, \end{cases}$$

ALGORITHM *RightLeftBinaryExponentiation*($a, b(n)$)

// right-to-left binary exponentiation algorithm ile a^n hesaplanır

// Input: a ve n pozitif sayısının ikili açılımındaki

// ikili dijitalerin $b_I, \dots, b_0$ listesi

// Output: a^n değeri

$term \leftarrow a$ //initializes a^{2^i}

if $b_0 = 1$ $product \leftarrow a$

else $product \leftarrow 1$

for $i \leftarrow 1$ **to** I **do**

$term \leftarrow term * term$

if $b_i = 1$ $product \leftarrow product * term$

return $product$

$$a^{13} \quad n = 13 = 1101_2.$$

$$13 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0.$$

1	1	0	1	n'in ikili digitleri
a^8	a^4	a^2	a	terms a^{2^i}
$a^5 \cdot a^8 = a^{13}$	$a \cdot a^4 = a^5$		a	çarpımlar

Algoritmanın verimliliği de soldan sağa ikili çarpmanın aynı nedenden ötürü logaritmiktir.

```
ALGORITHM  RightLeftBinaryExponentiation(a, b(n))
// right-to-left binary exponentiation algorithm ile a^n hesaplanır
//Input: a ve n pozitif sayısının ikili açılımındaki
//      ikili dijitlelerin b_1, ..., b_0 listesi
//Output: a^n değeri
term ← a //initializes a^{2^i}
if b_0 = 1 product ← a
else product ← 1
for i ← 1 to l do
    term ← term * term
    if b_i = 1 product ← product * term
return product
```

$$13 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0.$$
$$n = 13 = 1101_2.$$
$$13 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0.$$

$$a^{13}$$

1	1	0	1	n'in ikili digitleri
a^8	a^4	a^2	a	terms a^{2^i}
$a^5 \cdot a^8 = a^{13}$	$a \cdot a^4 = a^5$		a	çarpımlar

$\overset{1}{a^{16}}$	$\overset{1}{a^8}$	$\overset{1}{a^4}$	$\overset{1}{a^2}$	$\overset{0}{a}$	n'in ikili digitleri
$a^{16} \cdot a^{14} = a^{30}$	$a^6 \cdot a^8 = a^{14}$	$a^2 \cdot a^4 = a^6$	a^2	a	terms a^{2^i}
				1	çarpımlar

ALGORITHM *RightLeftBinaryExponentiation(a, b(n))*
 // right-to-left binary exponentiation algorithm ile a^n hesaplanır
 //Input: a ve n pozitif sayısının ikili açılımındaki
 // ikili dijitalerin $b_1, \dots, b_0$ listesi
 //Output: a^n değeri
 $term \leftarrow a$ //initializes a^{2^i}
if $b_0 = 1$ $product \leftarrow a$
else $product \leftarrow 1$
for $i \leftarrow 1$ **to** l **do**
 $term \leftarrow term * term$
 if $b_i = 1$ $product \leftarrow product * term$
return $product$

ALGORITHM *LeftRightBinaryExponentiation*($a, b(n)$)

// left-to-right binary exponentiation algorithm ile a^n hesaplanır

//Input: a ve n pozitif sayısının ikili açılımındaki

// ikili dijitalerin $b_1, \dots, b_0$ listesi

//Output: a^n değeri

$product \leftarrow a$

for $i \leftarrow 1 - 1$ **downto** 0 **do**

$product \leftarrow product * product$

if $b_i = 1$ $product \leftarrow product * a$

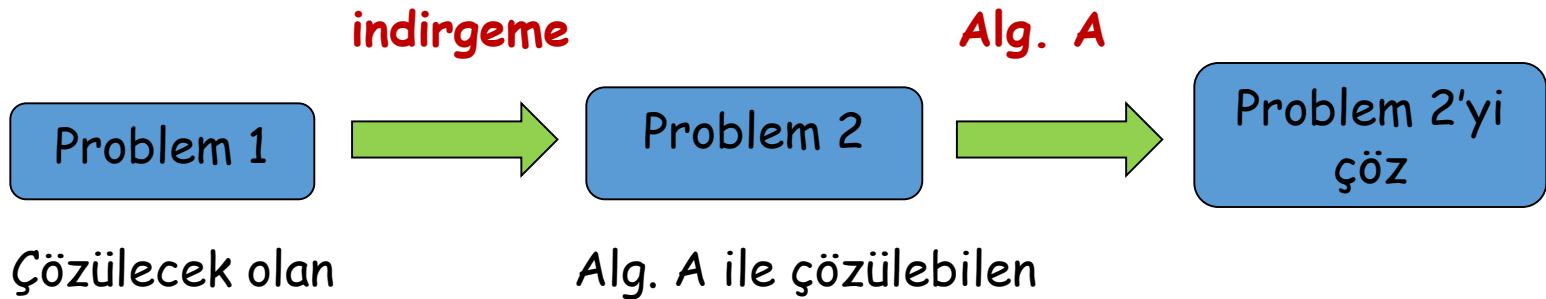
return $product$

~~x^{60}~~

60 = 111100

1	1	1	1	0	0
a	$a^2 \cdot a = a^3$	$(a^3)^2 \cdot a = a^7$	$(a^7)^2 \cdot a = a^{15}$	$a^{15} \cdot a^{15} = a^{30}$	$a^{30} \cdot a^{30} = a^{60}$

Trns. & Conq.: Problem İndirgeme



- İki pozitif tam sayının en küçük ortak çarpanı $\text{lcm}(m, n)$

- $m = 24$, ve $n = 60$

Öyleyse $m * n = \text{lcm}(m, n) * \text{gcd}(m, n)$

- $24 = 2 * 2 * 2 * 3$ ve $60 = 2 * 2 * 3 * 5$

$$\text{lcm}(m, n) = \frac{m * n}{\text{gcd}(m, n)} !$$

- m ve n 'nin ortak çarpanlarının çarpımını al ve n 'de olmayan m 'nin çarpanlarını ve m 'de olmayan n 'in çarpanlarını al
- $\text{lcm}(24, 60) = (2 * 2 * 3) * 2 * 5 = 120$

İyi bir algoritma mı?

Dikkat: $24 * 60 = (2 * 2 * 2 * 3) * (2 * 2 * 3 * 5) = (2 * 2 * 3)^2 * 2 * 5$